# TFE4152 Term Project

June 8, 2026

# 1 Introduction

***- Welcome to our headquarters! And thank you for being able to come here at such a short notice! We've had some big projects come in lately, and find ourselves in a bit of a pickle as our lead engineer is on a three month long vacation to New Zealand with her wife. Now we have no way of finishing our new chip before the tapeout deadline in November! Unless the two of you can help us out? I've heard such great things about you from your professors at NTNU.***

You look at each other and nod in agreement.

***- Wonderful! You'll be trusted with creating two crucial parts of our new chip. I'll email you the details.***

| | |
|---|---|
| From: | ingcognito@email.com |
| To: | icdesigners@tfe4152.ntnu.no |
| Date: | September 29th 2025 |
| Subject: | Project description |

Hei! As promised, here are more details on the project.

Firstly, we need a wide-swing current mirror. We use a 180 nm transistor technology, and create all our designs in AIMSpice. I've attached more information about specs (see section 2), please read and follow that carefully!

Secondly, our digital design team has been struggling to solve a problem for some time. Fortunately, our team lead sent us a solution idea right before boarding the plane! It involved something called a Tsetlin machine, which you can choose to read more about here `https://www.literal-labs.ai/logical-ai/` and here `https://tsetlinmachine.org/`. Your professor told me that Tsetlin machines were also covered in a guest lecture 27/8/2025. All the details she sent us regarding specifications etc. are included in the attached document (see section 3).

**This is an ongoing project, so as I'm sure you understand there may be updates to the spec at a later date.** If that is the case, you will be told through an announcement on your TFE4152 Blackboard page.

Thank you for helping us out!

Best regards,
Ing. Cognito

# 2 Analog circuit design

Cascode current mirrors offer higher output impedance than simple current mirrors, and are therefore preferred in circuits where high output impedance is crucial. Unfortunately, this desired property of the conventional cascode current mirrors comes at the cost of a limited output voltage swing.

We need you to design a current mirror that offers higher output impedance, while still allowing a wide output swing. Your task is therefore to design a *wide-swing current mirror*.

The design has to meet the following specifications:

- Transistor technology: 180 nm (`https://analogicdesign.com/students/netlists-models/`)
- Design for low power consumption.
- Support an input current $I_{in}$ ranging from 40 uA to 50 uA.
- This is a current mirror, so the output current $I_{out}$ should follow the input current $I_{in}$. A small dc offset is allowable (but not desirable). The most important thing is that the relationship is linear ($I_{in}$ increased by $x$ => $I_{out}$ increased by $x$).

You should aim to meet the circuit specifications for all the PVT-variations (see subsection 2.3).

To ensure that everything works as intended, your design must be properly tested. Use AIMSpice to implement and simulate your circuit.

## 2.1 Current Mirror Design

Figure 1 demonstrates a cascode current mirror.

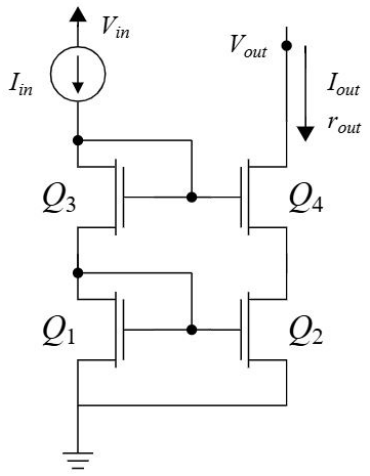


Figure 1: Schematic of a cascode current mirror

The figure demonstrates four NMOS transistors $Q_1$, $Q_2$, $Q_3$, and $Q_4$. Additionally, there is a current source $I_{in}$ and the mirrored current $I_{out}$. $I_{out}$ is joined by $r_{out}$ with an arrow to demonstrates from where the output current and resistance is measured from. $V_{in}$ and $V_{out}$ are the input and output voltages.

This can easily be proven to function as a current mirror. Figure 1 demonstrates that

$$V_{DS3} = V_{GS3} \tag{1}$$

$$V_{DS1} = V_{GS1} \tag{2}$$

where $V_{GSi}$ is the gate-source voltage, and $V_{DSi}$ is the drain-source voltage of $Q_i$. This means the transistors $Q_3$ and $Q_1$ are safely in the active region, and we can assume the square law model for the current running through them. The square law model is given by

$$I_D = \frac{1}{2}\mu_n C_{ox}\frac{W}{L}(V_{GS} - V_{tn})^2 = \frac{1}{2}\mu_n C_{ox}\frac{W}{L}V_{eff}^2 \tag{3}$$

where $I_D$ is the drain current through a NMOS transistor. For a description of the other parameter in (3), the reader is referred to Table 1.

Table 1: Definition of parameters in (3)

| Variable | Description |
|---|---|
| $I_D$ | Drain current [A] |
| $\mu_n$ | Electron mobility [$\text{cm}^2/\text{V}\cdot\text{s}$] |
| $C_{ox}$ | Gate oxide capacitance per unit area [$\text{F}/\text{cm}^2$] |
| $W$ | Transistor channel width [m] |
| $L$ | Transistor channel length [m] |
| $V_{GS}$ | Gate-to-source voltage [V] |
| $V_{tn}$ | Threshold voltage of NMOS transistor [V] |
| $V_{eff}$ | Effective voltage, $V_{GS} - V_{tn}$ [V] |

$I_{in}$ ensures that $V_{GS3}$ and $V_{GS1}$ are set to pass the current it provides. As all transistors in Figure 1 are identical, (3) shows that

$$V_{GS1} = V_{GS3}. \tag{4}$$

Since both the source of $Q_1$ and $Q_2$ are grounded, we also know

$$V_{GS1} = V_{GS2} \tag{5}$$

meaning $Q_2$ and $Q_1$ lead the same current. Naturally, the current through $Q_4$ must match the one through $Q_2$, and therefore

$$V_{GS1} = V_{GS2} = V_{GS3} = V_{GS4}. \tag{6}$$

$I_{out}$ is therefore mirroring the current $I_{in}$, as $V_{GS3}$ and $V_{GS1}$ ensure $I_{in}$, and $V_{GS4}$ and $V_{GS2}$ reflect the aforementioned gate-source voltages.

An advantage with the cascode current mirror is the high output impedance. This can be found by setting up the small signal equivalence of Figure 1, as presented in Figure 2.

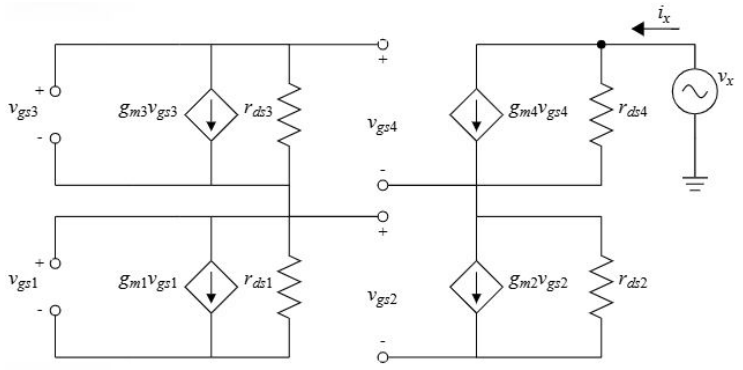


Figure 2: Schematic of the small signal cascode current

$v_{gsi}$ is the small signal gate-source voltage of transistor $Q_i$. Similarly, $g_{mi}$ is the transconductance and $r_{dsi}$ the small-signal resistance of transistor $Q_i$. $v_x$ is the applied voltage with current $i_x$ used to measure the output resistance, where

$$r_{out} = \frac{v_x}{i_x}. \tag{7}$$

Figure 2 can be simplified to

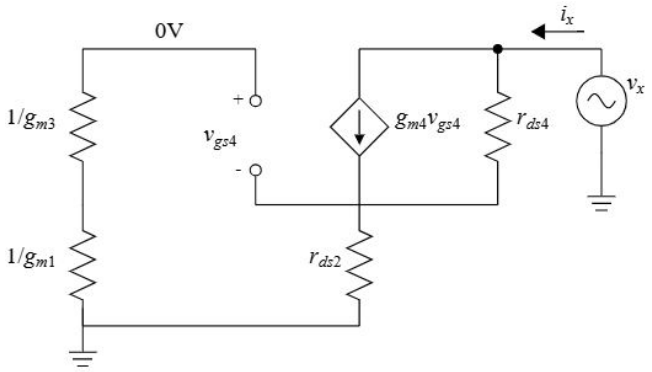


Figure 3: Schematic of the simplified small signal cascode current

where this simplification is done on the basis of the simplification presented in Figure 4, as explained in page 118 of (Carusone, et al., 2011, p. 118). In additon, the dependant current source $g_{m2}v_{gs2}$ is removed in Figure 3 as, when analyzing the output resistance, we know $v_{gs2} = 0$. This is because the gate becomes grounded under the analysis, and the source is already grounded (Carusone, et al., 2011, p. 119). This is also why $v_{g4}$ is grounded in Figure 3.

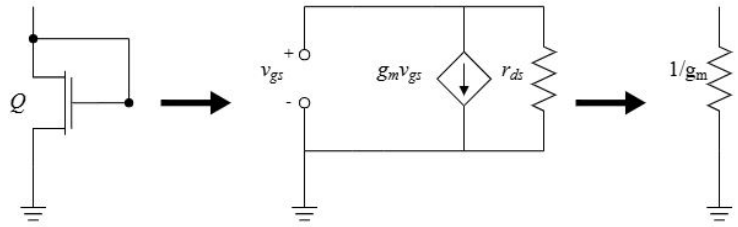


Figure 4: A small signal model for a diode-connected NMOS transistor $Q$

Figure 3 allows for the calculation of the output resistance $r_{out}$. We begin by finding $i_x$, which is equal to the total current through $g_{m4}v_{gs4}$ and $r_{ds4}$.

$$i_x = g_{m4}v_{gs4} + \frac{v_x - v_{s4}}{r_{ds4}} \tag{8}$$

$$i_x = -g_{m4} i_x r_{ds2} + \frac{v_x - i_x r_{ds2}}{r_{ds4}} \tag{9}$$

Using equation (7) by plugging in the value for $i_x$ from (9), we get

$$r_{out} = r_{ds4}[1 + r_{ds2}(g_{m4} + g_{ds4})] \tag{10}$$

where $g_{ds4}$ is equal to $1/r_{ds4}$. Still, Figure 3, and therefore (10), ignores the body effect of $Q_4$, even though $v_{s4}$ is not connected to ground. But since $v_{g4}$ is grounded in Figure 3, the body effect can be taken into account by simply replacing $g_{m4}$ in (10) with $g_{m4} + g_{s4}$, where $g_s$ is the transconductance between the source–bulk voltage in $Q_4$ (Carusone, et al., 2011, p. 125).

$$r_{out} = r_{ds4}[1 + r_{ds2}(g_{m4} + g_{s4} + g_{ds4})] \tag{11}$$

Further, since $g_{m4} \gg g_{s4} \gg g_{ds4}$ and $r_{ds2} \gg 1$, we have

$$r_{out} \simeq r_{ds4}[r_{ds2}(g_{m4})]. \tag{12}$$

This output resistance can run up to a few megaohm, as will be shown when we calculate the $r_{out}$ of our system. Yet the problem with the cascode current mirror is it does not allow for a wide output swing. This is shown by the result in (6), that all $V_{GSi}$ are equal to one another in Figure 1. This means all the transistors have an equal effective voltage $V_{eff}$, as

$$V_{GS} = V_{eff} + V_{tn} \tag{13}$$

where $V_{tn}$ is the threshold voltage of a NMOS transistor. Naturally, all transistors in Figure 1 have the same $V_{tn}$ as the current mirror is built with identical transistors. From this we can deduce that

$$V_{G3} = V_{G1} + V_{GS3} \tag{14}$$

$$V_{G3} = V_{GS1} + V_{GS3} \tag{15}$$

$$V_{G3} = 2V_{eff} + 2V_{tn} \tag{16}$$

and

$$V_{DS2} = V_{S4} \tag{17}$$

$$V_{DS2} = V_{G3} - V_{GS4} \tag{18}$$

$$V_{DS2} = (2V_{eff} + 2V_{tn}) - (V_{eff} + V_{tn}) \tag{19}$$

$$V_{DS2} = V_{eff} + V_{tn}. \tag{20}$$

As we know that for $V_{DS4}$ to be in the active region, it must

$$V_{DS4} = V_{out} - V_{S4} > V_{eff} \tag{21}$$

meaning $V_{out}$ must be larger than

$$V_{out} > V_{eff} + V_{S4} \tag{22}$$

$$V_{out} > V_{eff} + V_{DS2} \tag{23}$$

$$V_{out} > 2V_{eff} + V_{tn} \tag{24}$$

which is $V_{tn}$ greater than the minimum value $V_{out}$ potentially could have been (Carusone, et al., 2011, p. 130). Due to this loss of output swing, a variant of the cascode mirror will be designed, as presented in Figure 5.

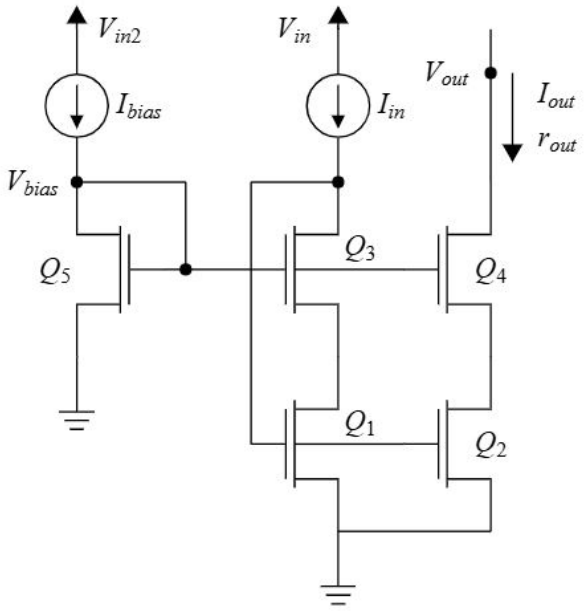


Figure 5: Schematic of the wide-swing cascode current mirror

This variant is referred to as a wide-swing cascode current mirror. Figure 5 introduces a new NMOS transistor $Q_5$, along with the current source $I_{bias}$. The role of of these components is to provide the gate voltage $V_{bias}$ to $Q_3$ and $Q_4$. When $I_{bias}$ is equal to $I_{in}$, and the dimensions of the transistors are as given in Table 2, we can prove that the output swing maximizes.

Table 2: Transistor dimensions for the wide-swing cascode current mirror

| **Transistor** | **Size** |
|---|---|
| $Q_1$ | W/L |
| $Q_2$ | W/L |
| $Q_3$ | $\frac{W/L}{n^2}$ |
| $Q_4$ | $\frac{W/L}{n^2}$ |
| $Q_5$ | $\frac{W/L}{(n+1)^2}$ |

Before proving that the output swing is maximized, note that the output resistance of Figure 5 is the same as the output impedance in (10), as Figure 5 and Figure 1 demonstrate identical systems when looking in from the output arrows of both figures.

The output swing can be found in a similar fashion to how it was found for the cascode current mirror. As $I_{bias}$ and $I_{in}$ are equal, and $I_{out}$ mirrors $I_{in}$, the current through each transistor is equal. An equation for $V_{eff}$ can then be found by rearranging (3) such that

$$V_{eff} = V_{eff1} = V_{eff2} = \sqrt{\frac{2I_{D2}}{\mu_n C_{ox}(W/L)}} \tag{25}$$

where $V_{effi}$ is the effective voltage of transistor $Q_i$, and $I_{D2}$ is the drain current through $Q_2$, and therefore effectively $Q_1$ too. Since $Q_5$ has the same drain current, but is $(n+1)^2$ times smaller, we have

$$V_{eff5} = (n+1)V_{eff}. \tag{26}$$

Similarly, we get

$$V_{eff3} = V_{eff4} = nV_{eff}. \tag{27}$$

Now we can

$$V_{G3} = V_{G4} = V_{GS5} = (n+1)V_{eff} + V_{tn} \tag{28}$$

where one must remember (13). Further,

$$V_{DS1} = V_{DS2} = V_{S4} = V_{G5} - V_{GS4} = V_{GS5} - (nV_{eff} + V_{tn}) = V_{eff}. \tag{29}$$

Again, for $V_{DS4}$ to be in the active region we must have

$$V_{DS4} = V_{out} - V_{S4} > V_{eff4} \tag{30}$$

$$V_{out} > (n+1)V_{eff} \tag{31}$$

where if we set n=1

$$V_{out} > 2V_{eff} \tag{32}$$

meaning we have minimized $V_{out}$ by a whole threshold voltage from the simple cascode current mirror, as shown in (24).

Therefore, the wide-swing current mirror is a great current mirror that offers high output impedance while allowing a wide output swing. The design for low power consumption will be discussed in the simulations of the current mirror, in subsection 2.3. The total power consumption of Figure 5 is

$$P = I_{bias}V_{in2} + I_{in}V_{in} + I_{out}V_{out}. \tag{33}$$

## 2.2 Implementation

The wide-swing cascode current mirror from Figure 5 was implemented in AIM-Spice (AIM-Spice, 2020) as presented in the code below.

Listing 1: AIM-Spice implementation of the wide-swing cascode current mirror

```
1  Project wide swing amplifier
2
3  .include p18_cmos_models_tt.inc
4
5  Vin2 1 0 DC 1V
6  Vin 2 0 DC 1V
7  Vout out 0 DC 1V
8
9  Ibias 1 3 50u
10 Iin 2 4 50u
11
12 * Current mirror reference transistor
13 MQ5 3 3 0 0 NMOS W=0.36u L=0.18u
14
15 * Cascode stage transistors
16 MQ3 4 3 5 5 NMOS W=1.44u L=0.18u
17 MQ1 5 4 0 0 NMOS W=1.44u L=0.18u
18
19 * Output stage cascode transistors
20 MQ4 out 3 7 7 NMOS W=1.44u L=0.18u
21 MQ2 7 4 0 0 NMOS W=1.44u L=0.18u
22
23 .plot dc id(MQ4) id(MQ2)
```

The code begins by including a model files for 180nm CMOS technology, such that the transistors declared later in the code follow this technology. This model can be found and downloaded from (Learning Microelectronics, n.d.). It continues by introducing $V_{in2}$, $V_{in}$ and $V_{out}$ as presented in Figure 5. Further, $I_{bias}$ and $I_{in}$ are introduced, and the transistors $Q_1$ to $Q_5$ as $MQ_1$ to $MQ_5$. The final line is how the mirrored current was plotted against a sweeping $V_{out}$.

Lines 13-21 show that the length of the transistors were chosen to be 180 nm. Following the layout of the smallest possible CMOS transistor (Carusone, et al., 2011, p. 87) and allowing n=1 for the values presented in Table 2, transistor $MQ_5$ must have a width two times larger than the length at 360nm, and the rest become four times larger than that at 1440nm. In subsection 2.3 it will be discussed why larger transistors should have been chosen to make the circuit more power efficient.

With these values, $r_{out}$ can be calculated using (12). If we accept $I_{out} = I_{in}$ and that we are operating in the active region, we get that

$$g_{m4} = \frac{\partial I_{out}}{\partial V_{GS4}} = \mu_n C_{ox} \frac{W}{L} V_{eff4} \tag{34}$$

where the equation for $I_{out}$ is (3). As n=1, $V_{eff4}$ is equal to $V_{eff}$ as shown in (27). The equation for $V_{eff}$ from (25) can then be used so

$$g_{m4} = \sqrt{2\mu_n C_{ox}\left(\frac{W}{L}\right) I_{D2}} \tag{35}$$

$$g_{m4} = \sqrt{2\mu_n C_{ox}\left(\frac{W}{L}\right) I_{out}}. \tag{36}$$

Using Table 1.5 provided in page 53 of (Carusone, et al., 2011, p. 53), we get the values for $\mu_n C_{ox}$ in 180nm CMOS technology. We get

$$g_{m4} = \sqrt{2 * 0.27 * \left(\frac{1.44}{0.18}\right) I_{out}} \text{ mA/V} \tag{37}$$

$$g_{m4} = 2.08\sqrt{I_{out}} \text{ mA/V}. \tag{38}$$

We also have

$$r_{ds4} = r_{ds2} = \frac{1}{\lambda I_{out}} = \frac{L}{\lambda L I_{out}} = \frac{0.18}{0.08 I_{out}} \text{ k}\Omega = 2.25\frac{1}{I_{out}} \text{ k}\Omega \tag{39}$$

where $I_{out}$ is given in mA in both (38) and (39). $\lambda$L value is also found in Table 1.5 of (Carusone, et al., 2011, p. 53), where $\lambda$ is the output impedance constant (Carusone, et al., 2011, p. 22). $r_{out}$ is then

$$r_{out} = \frac{2.25^2}{I_{out}^2} * 2.08\sqrt{I_{out}} \text{ k}\Omega \tag{40}$$

$$r_{out} = \frac{10.52}{\sqrt{I_{out}^3}} \text{ k}\Omega. \tag{41}$$

Since we will be testing currents from 40$\mu$A to 50$\mu$A, our range of output resistance will be

$$r_{out-max} = \frac{10.52}{\sqrt{0.04^3}} \text{ k}\Omega = 1315.28 \text{ k}\Omega \tag{42}$$

$$r_{out-min} = \frac{10.52}{\sqrt{0.05^3}} \text{ k}\Omega = 941.14 \text{ k}\Omega. \tag{43}$$

The minimum $V_{out}$ can also be found by finding $V_{eff}$, as show in in (32). $V_{eff}$ is found using (25) such that

$$V_{eff-50\mu A} = \sqrt{\frac{100}{270(\frac{1.44}{0.18})}} \tag{44}$$

$$V_{eff-50\mu A} = 0.22 \text{ V} \tag{45}$$

$$V_{out-50\mu A} > 2 * 0.22 \text{ V} = 0.44 \text{ V} \tag{46}$$

where $V_{out-50\mu A}$ the minimum output voltage when mirroring a current of $50\mu$A. Similarly, $V_{out-40\mu A}$ is the minimum output voltage when mirroring a current of $40\mu$A and is found by

$$V_{eff-40\mu A} = \sqrt{\frac{80}{270(\frac{1.44}{0.18})}} \tag{47}$$

$$V_{eff-40\mu A} = 0.19 \text{ V} \tag{48}$$

$$V_{out-40\mu A} > 2 * 0.19 \text{ V} = 0.38 \text{ V}. \tag{49}$$

## 2.3 Circuit Simulation Results

We can simulate the current mirror under varying conditions by modifying the code described in Listing 1. Under this subsection, $V_{DD}$ refers to both $V_{in}$ and $V_{in2}$. $T_{op}$ refers to the temperature the circuit is operating under.

### 2.3.1 Simulation of Power Consumption

Listing 1 was made to plot $I_{out}$ under varying transistor sizes by changing the length and width of the transistors in Lines 13-21. This was done to simulate power consumption when the current mirror has varying output resistance. Figure 6 presents the result, where $I_{in}$ and $I_{bias}$ in Line 9-10 were also changed to supply a current of 45$\mu$A.

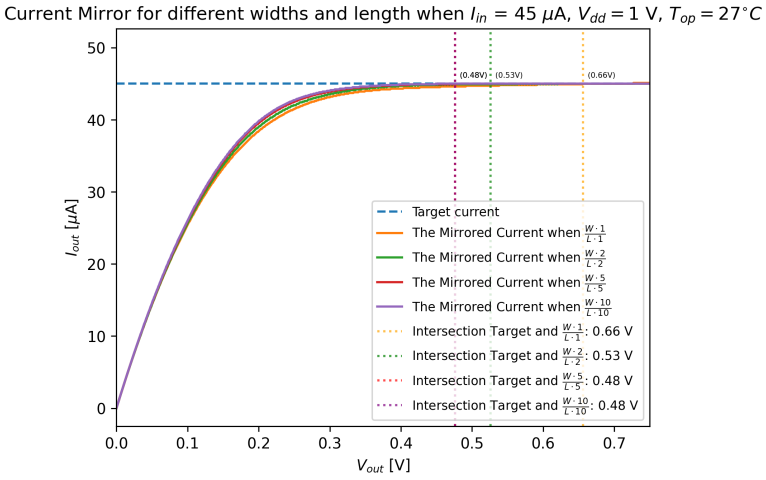


Figure 6: The current mirror under different transistor sizes

$W$ and $L$ in Figure 6 refer to the width and length of transistors $MQ_1$ to $MQ_5$. As we can see, the larger transistors mirror the current at a lower $V_{out}$, meaning the circuits with larger transistors have a smaller power consumption. Using the values from the intersection targets in Figure 6 and (33), the highest power consumption with the smallest transistor size is

$$P_{high} = (3 * 0.66)\,\text{V} * (45 * 10^{-6})\,\text{A} \tag{50}$$

$$P_{high} = 89.1\ \mu\text{W} \tag{51}$$

where not only $V_{out}$, but also $V_{in2}$ and $V_{in}$ are all set to 0.66V. This is because we now know we can change the input voltages in Line 5-6 to be 0.66V and have the $I_{bias}$ and $I_{in}$ provide the correct current.

Similarly, the lowest power consumption with the highest transistor size is

$$P_{low} = (3 * 0.48)\,\mathrm{V} * (45 * 10^{-6})\,\mathrm{A} \tag{52}$$

$$P_{low} = 64.8\ \mu\mathrm{W}. \tag{53}$$

The output resistance of each current mirror was plotted, as presented in Figure 7. This was done by changing Line 23 to two lines

```
.plot dc rds(MQ4) rds(MQ2)
```
(54)

```
.plot dc gm(MQ4)
```
(55)

and exporting the new plots as a CSV file. We then had a datafile containing all the information in (12), allowing us to calculate and plot $r_{out}$ under a sweeping $V_{out}$ with a data processing code from Python. Figure 7 demonstrates the resulting plot.

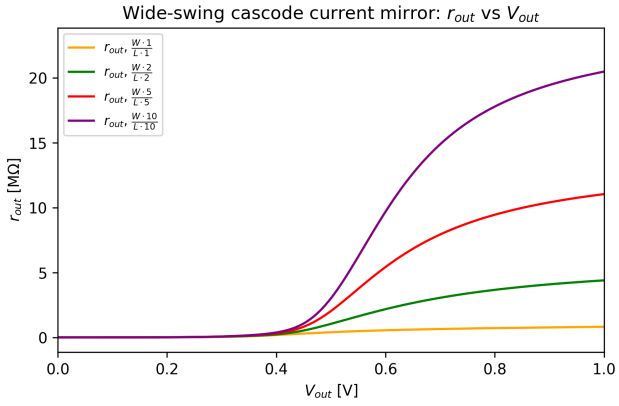


Figure 7: Output resistance under different transistor sizes

As expected, larger transistors lead to a larger output resistance. This is expected as $\lambda$ from (39) is smaller for larger lengths of transistors (Carusone, et al., 2011, p. 53). Larger resistance and smaller power consumption shows that the length and width of the transistors in Listing 1 should have been larger.

### 2.3.2 Simulation of Process Corner Variations

The current mirror was tested under the different process corner variations TT (Typical, Typical), FF (Fast, Fast) and SS (Slow, Slow) (Weste & Harris, 2010, p. 244). This was done by changing Line 3 of Listing 1 to include the correct model file, where the tt is changed to ff and ss. These files are found

when downloading the model for the 180nm technology transistors (Learning Microelectronics, n.d.). For each process variation, we have $I_{in}$ and $I_{bias}$ be 40 $\mu$A, 45 $\mu$A, and 50 $\mu$A by changing Lines 9-10. Figure 8 demonstrates the plot for $I_{out}$ when mirroring a current of 40 $\mu$A under the three process variations.

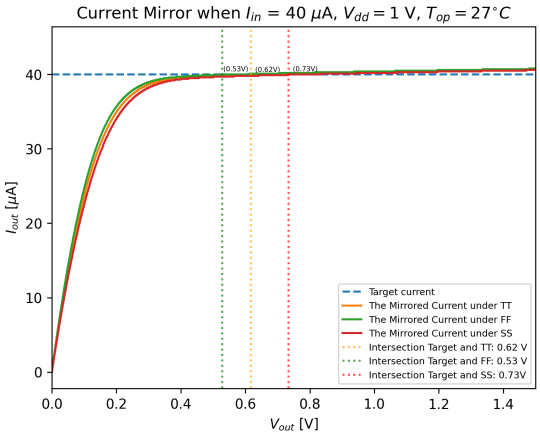


Figure 8: The current mirror mirroring 40 $\mu$A under TT, FF and SS

Figure 9 and 10 also demonstrate $I_{out}$ for the respective input currents of 45 $\mu$A and 50 $\mu$A.

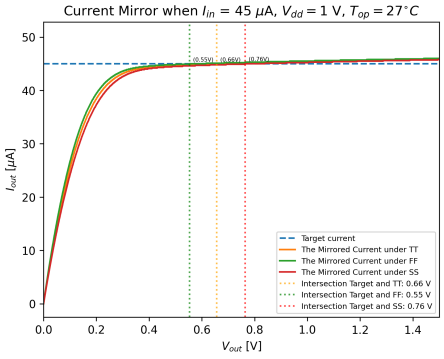


Figure 9: The current mirror mirroring 45 $\mu$A under TT, FF and SS

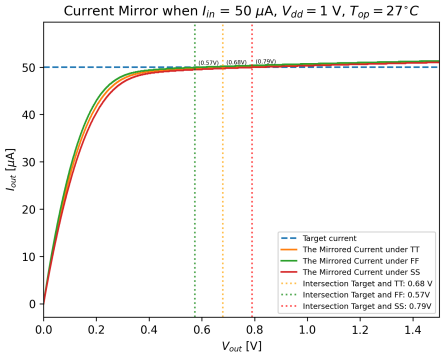


Figure 10: The current mirror mirroring 50 $\mu$A under TT, FF and SS

As the figures show, the faster the transistors, the lower of a $V_{out}$ is needed before the current is mirrored. This is expected, as faster transistors means $Q_5$ from Figure 5 provides $I_{bias}$ at a lower $V_{bias}$ (Weste & Harris, 2010, p. 245). This means a lower $V_{G4}$ and thereby a lower $V_{eff4} = V_{eff}$. So the faster the transistors, the active region is reached at a lower $V_{out}$, as shown in (32).

### 2.3.3 Simulation of Supply Voltage Variations

Listing 1 was changed to analyze the current mirror under a supply voltage variation of -10% to 10%. This was done by changing Lines 5-6 so $V_{in2}$ and $V_{in}$ provided 0.9V and 1.1V. Figure 11 demonstrates the current mirror before the voltage is altered with, under all input current.

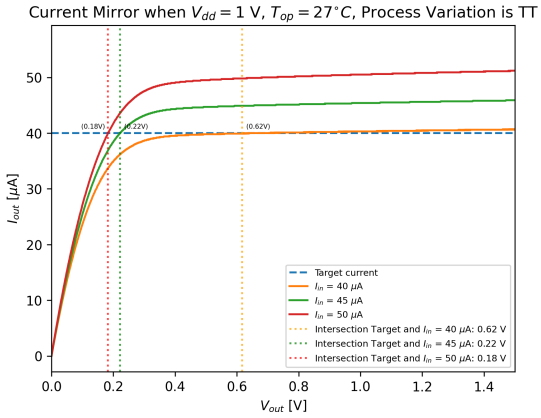


Figure 11: The current mirror mirroring under $V_{DD}$=1 V

Figure 12 and 13 demonstrate the same, but with the supply voltages now respectively being 0.9V and 1.1V.

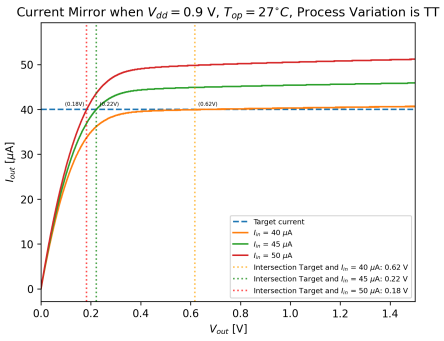


Figure 12: The current mirror mirroring under $V_{DD}$=0.9V

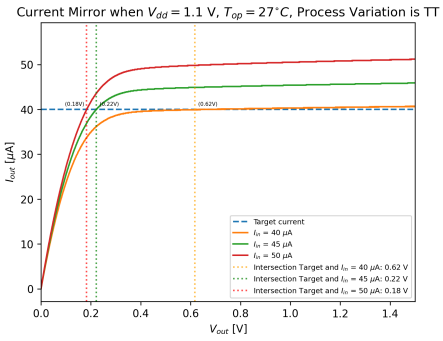


Figure 13: The current mirror mirroring under $V_{DD}$=1.1V

As can be seen with the intersection targets of the figures, there is no difference when varrying the voltages. This is explained by the current sources $I_{bias}$ and $I_{in}$. As we see in Figure 6 and was explained below (51), as long as $V_{in}$ and $V_{in2}$ are larger than 0.66V for our current mirror, the current sources will be able to provide the current they wish to. As our varying voltages are larger

than that, the current mirror reacts the same way in both cases.

### 2.3.4 Simulation of Temperature variations

Simulations of temperature variations are done by changing the operational temperature $T_{op}$. This is not done in Listing 1, but rather must be done within the AIM-Spice simulator. The circuit was simulated in temperatures of $T_{op} = 0°\text{C}$, $T_{op} = 27°\text{C}$ and $T_{op} = 50°\text{C}$, and was tested with all input currents. Figure 14 demonstrates the current mirror under 27°C.

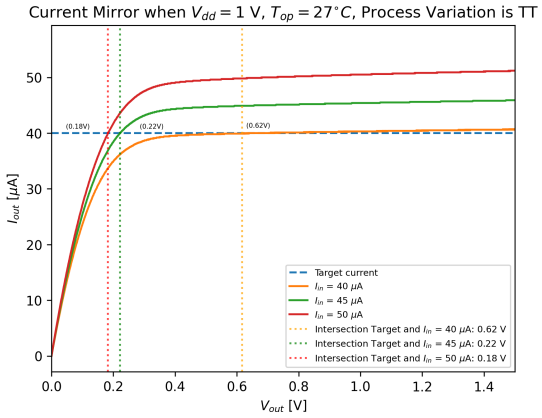


Figure 14: The current mirror under $T_{op} = 27°\text{C}$

Figure 15 and Figure 16 show the same, but respectively with $T_{op} = 0°\text{C}$ and $T_{op} = 50°C$.

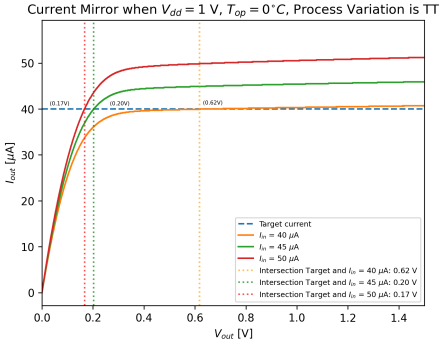


Figure 15: The current mirror mirroring under $T_{op} = 0°\text{C}$

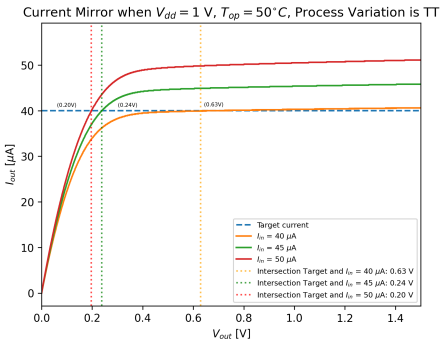


Figure 16: The current mirror mirroring under $T_{op} = 50°\text{C}$

As can be seen with the intersection targets, there is minimal difference in the plots. This is wished for, as it shows that our circuit is able to function as

intended under harsher temperatures.

## 2.4 Discussion

The simulations of the current mirror show that the circuit worked as intended. The current was mirrored once $V_{out}$ reached the active region, where $r_{out}$ was calculated to be large under subsection 2.3. Figure 7 shows a plot of different $r_{out}$ values, where it is found that larger transistors have larger $r_{out}$ values. For this reason, our realized current mirror should have utilized larger transistors. The minimum $V_{out}$ was also calculated to be small under subsection 2.3.

The circuit also functioned as wished for and expected under varying conditions. The power consumption was seen to be improved in Figure 6, by using larger transistors. Another reason why larger transistors should have been utilized rather than the sizes we used. The tradeoff becomes a more power efficient circuit, but with a greater area penalty.

As explained in page 266 of (Carusone, et al., 2011, p. 266), a wide-swing current mirror with enhanced output impedance could rather have been used to secure a larger output resistance, while the minimum $V_{out}$ remains as given in (32). The tradeoff is that the power dissipation almost doubles (Carusone, et al., 2011, p. 266), which is not wished for.

# 3 Digital Design - Creating a State Machine

*"The Tsetlin Machine is not just another machine learning algorithm. It draws on principles laid down by Mikhail Tsetlin, a visionary Soviet mathematician, who in the 1960s explored a radically different path to artificial intelligence. Rather than mimicking biological neurons, Tsetlin's approach was rooted in learning automata and game theory. He recognised that logic—expressed through what we now call Tsetlin automata—could classify data more efficiently, forging a new direction in AI."* (`https://www.literal-labs.ai/tsetlin-machines/`)

Design a two-action Tsetlin automaton. Since the Tsetlin automaton can be seen computationally as a finite state machine (FSM), it will be referred to as such in the rest of this document.

Here are the relevant specifications for your FSM:

- It should have 6 states: $\Phi = \{\phi_0, \phi_1, \phi_2, \phi_3, \phi_4, \phi_5\}$.
- Transition from one state to another is triggered by the input signal $\beta$, which can be either $\beta_{Penalty} = 0$ or $\beta_{Reward} = 1$.
- The expected transition for a state $\phi_u$, given an input $\beta_x$:
$$\phi_{next} = \begin{cases} \phi_{u-1}, \text{ if } 0 \leq u \leq 2 \text{ and } x = \text{Reward}, \\ \phi_{u+1}, \text{ if } 0 \leq u \leq 2 \text{ and } x = \text{Penalty}, \\ \phi_{u+1}, \text{ if } 3 \leq u \leq 5 \text{ and } x = \text{Reward}, \\ \phi_{u-1}, \text{ if } 3 \leq u \leq 5 \text{ and } x = \text{Penalty}, \\ \phi_u \text{ otherwise.} \end{cases}$$
- The output signal $\alpha$ will be either $\alpha_0 = 0$ or $\alpha_1 = 1$.
- Implement your finite state machine as a Moore Machine,
where $\alpha = \begin{cases} \alpha_0, \text{if } 0 \leq u \leq 2. \\ \alpha_1, \text{if } 3 \leq u \leq 5. \end{cases}$
- Design the FSM with low power consumption in mind. Both static and dynamic power consumption should be considered.

The FSM should be implemented in Verilog using ActiveHDL. You must write the verilog code on structural (logic gate) level, where all gates and their interconnects are identifiable. This is done to ensure that you actually simulate the circuit you have designed.

You do *not* have to write the testbenches in logic gate level Verilog. Here you are free to use high level constructions in Verilog, or simply the graphical user interface in ActiveHDL.

## 3.1 Construction of the State Machine

The state machine can be constructed by following a nine step method for designing a clocked sequential logic (Mano, 1991, p. 235). We will follow a seven step method to construct a finite state machine, based on the description and specifications provided in the previous page. Each step in the seven step method is described below.

### Step 1: Derive the State Diagram

The state diagram, that was derived from the descriptions and specifications of desired operations in the beginning of section 3, is shown in Figure 17.

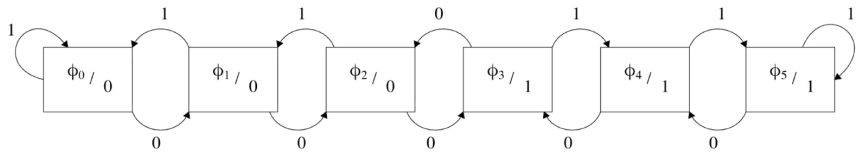


Figure 17: State diagram derived from specifications of desired operation.

The state diagram was obtained by first fulfilling the first specification, that the finite state machine should have six states. The six states are shown as squares in the state diagram. Then, looking at the expected transition for a state $\phi_u$, given an input $\beta_x$, we can draw arrows between the states indicating transition from one state to next state. Lastly, the output signal $\alpha_x$ is taken into consideration. The finite state machine is specified to be a Moore Machine, meaning the output should only be based on current state, unlike the Mealy Machine which has output based on both input signal and current state (Weste & Harris, 2010, p. 735). Whether the output signal is $\alpha_0 = 0$ or $\alpha_1 = 1$ is shown in each respective state $\phi_u$ in the state diagram.

The specification of designing the finite state machine with low power consumption is not taken into account in this step, but is discussed in step 7.

### Step 2: State Reduction

No state reduction was necessary, as the design already required six distinct states according to the specifications.

**Step 3: Assign Binary Values to The States**

Binary values were assigned to the states. It is necessary to have a three bit binary code since there is six states, as it would not have been possible to represent all six states with only a two bit binary code. The initial assignment of binary values to the states, starting at 000 and adding 1 in binary, is shown in Table 3 below.

Table 3: Initial assignment of binary values to states.

| **State** | **Binary value** |
|---|---|
| $\phi_0$ | 000 |
| $\phi_1$ | 001 |
| $\phi_2$ | 010 |
| $\phi_3$ | 011 |
| $\phi_4$ | 100 |
| $\phi_5$ | 101 |

During analysis of Table 3, and considering low power consumption, it was observed that transitioning from $\phi_3 = 011$ to $\phi_4 = 100$ (when $\beta_{reward} = 1$) would require flipping three bits. With this in mind and looking at the formula for dynamic power

$$P_{dyn} = \alpha C_L V_{dd}^2 f \tag{56}$$

where $\alpha$ is an activity factor which gives information about the average switching of bits during a transition from different states, and $f$ is the frequency of a clock. $C_L$ is then an effective capacitance that must be charged and discharged when a signal changes value, and will be discussed later in Step 7 (Weste & Harris, 2010, p. 43). Reducing the amount of bit flips when transitioning from $\phi_3$ to $\phi_4$ will reduce the activity factor $\alpha$ and will lead to reduction in dynamic power consumption. Thus, the state $\phi_3$ was reassigned from 011 to 100, $\phi_4$ from 100 to 101 and lastly $\phi_5$ from 101 to 110, to construct the finite state machine with low power consumption in mind. Final assignment of binary values to the states is shown in Table 4.

Table 4: Final assignment of binary values to states.

| **State** | **Binary value** |
|---|---|
| $\phi_0$ | 000 |
| $\phi_1$ | 001 |
| $\phi_2$ | 010 |
| $\phi_3$ | 100 |
| $\phi_4$ | 101 |
| $\phi_5$ | 110 |

The state diagram from Figure 17 with binary values for the states is shown in Figure 18.

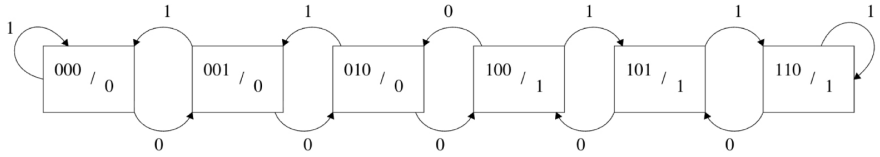


Figure 18: State diagram with binary values.

It is now possible to derive a binary-coded state table with the state diagram above.

**Step 4: Obtain the Binary-Coded State Table**

The binary-coded state table was derived directly from the state diagram in Figure 18. Each row represents a transition from present state to next state for a given input, $\beta_x$, as well as the corresponding output $\alpha_x$. Output $\alpha_x$ has nothing to do with the activity factor $\alpha$ from equation of dynamic power (56). The binary-coded state table is shown in Table 5 below.

Table 5: Binary-coded state table.

| **Present MSB** | **Present Mid** | **Present LSB** | **Input** | **Next MSB** | **Next Mid** | **Next LSB** | **Output** |
|---|---|---|---|---|---|---|---|
| A | B | C | $\beta$ | A | B | C | $\alpha$ |
| 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | x | x | x | x |
| 0 | 1 | 1 | 1 | x | x | x | x |
| 1 | 0 | 0 | 0 | 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 1 | 0 | 1 | 1 |
| 1 | 0 | 1 | 0 | 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 1 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 0 | x | x | x | x |
| 1 | 1 | 1 | 1 | x | x | x | x |

As seen in the table, the bits in the three bit binary code are denoted as $A$, $B$ and $C$. $A$ corresponds to the most significant bit (MSB), which is the leftmost bit of the binary state. $B$ corresponds to the middle bit and $C$ is the least significant bit (LSB), which is the rightmost bit. The table was derived by looking at current state in the state diagram and following the arrows in Figure 18, whether input is $\beta_{penalty} = 0$ or $\beta_{reward} = 1$, to see what the next state is and if output is $\alpha_0 = 0$ or $\alpha_1 = 1$.

**Step 5: Choose Type of Flip-Flops**

Master-slave D flip-flops were selected for the design (Weste & Harris, 2010, p. 17), as they are are well suited for synchronous sequential circuits where state changes occur on the rising edge of the clock. A better explanation of the Master-slave D flip-flop comes in step 7. A register with three D flip-flops were used, one for each bit in the binary bit code.

**Step 6: Derive Simplified Flip-Flop and Output Equations**

The simplified input equations for the flip-flops and system output were derived using the state table. We let $D_A$, $D_B$ and $D_C$ represent the flip-flop input equations for each respective bit A, B and C. Output is still represented by $\alpha$. From the state table, Karnaugh maps were drawn for each flip-flop input and for the system output function. The Karnaugh map for the output equation $\alpha$ is shown in Figure 19.

| $AB$ \ $C\beta$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 0 | 0 | 0 | 0 |
| 01 | 0 | 0 | X | X |
| 11 | 1 | 1 | X | X |
| 10 | 1 | 1 | 1 | 1 |

Figure 19: Karnaugh map for the output $\alpha$.

From the Karnaugh map above and simple boolean algebra (Kartik, 2025), it is possible to show that the equation for $\alpha$ is given as

$$\alpha = A\overline{B}\,\overline{C} + AB\overline{C} + A\overline{B}C. \tag{57}$$

Same process is repeated to find flip-flop input equations. Each respective Karnaugh map is shown in Figure 20.

| $AB$ \ $C\beta$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 0 | 0 | 0 | 0 |
| 01 | 1 | 0 | X | X |
| 11 | 1 | 1 | X | X |
| 10 | 0 | 1 | 1 | 1 |

(a) K-map for $D_A$

| $AB$ \ $C\beta$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 0 | 0 | 0 | 1 |
| 01 | 0 | 0 | X | X |
| 11 | 0 | 1 | X | X |
| 10 | 1 | 0 | 1 | 0 |

(b) K-map for $D_B$

| $AB$ \ $C\beta$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 1 | 0 | 0 | 0 |
| 01 | 0 | 1 | X | X |
| 11 | 1 | 0 | X | X |
| 10 | 0 | 1 | 0 | 0 |

(c) K-map for $D_C$

Figure 20: Karnaugh maps for the flip-flop input equations $D_A$, $D_B$, and $D_C$.

The maps are again used to obtain minimal Boolean expression. An overview of every boolean expression is shown in Table 6.

Table 6: Overview of derived Boolean expressions

| **Signal** | **Expression** |
|---|---|
| $\alpha$ | $A\overline{B}\,\overline{C} + AB\overline{C} + A\overline{B}C$ |
| $D_A$ | $B\overline{C}\,\overline{\beta} + A\overline{C}\beta + A\overline{B}C$ |
| $D_B$ | $\overline{A}\,\overline{B}C\overline{\beta} + AB\overline{C}\beta + A\overline{B}\,\overline{C}\,\overline{\beta} + A\overline{B}C\beta$ |
| $D_C$ | $\overline{A}\,\overline{B}\,\overline{C}\,\overline{\beta} + \overline{A}B\overline{C}\beta + AB\overline{C}\,\overline{\beta} + A\overline{B}\,\overline{C}\beta$ |

**Step 7: Draw the Logic Diagram**

Using the simplified equations, it is now possible to draw a final logic diagram. Figure 21 shows how a general state machine is constructed.

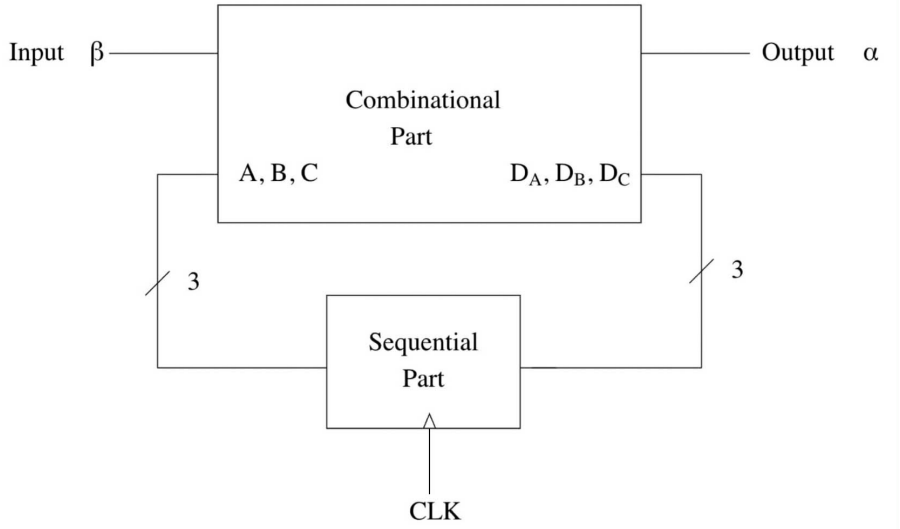


Figure 21: Construction of general state machine.

The figure shows a combinational part, which is the part that has been discussed up until now, and a sequential part that stores bit values from the combinational part. The sequential part will be discussed later in this step. Before drawing the combinational logic diagram, we can now consider the specification of designing the finite state machine with low power consumption. The first thing to consider is looking at each logic gate at transistor level. In CMOS, each logic gate is implemented using transistors and total number of transistors will directly affect both static and dynamic power consumption (Weste & Harris, 2010, p. 55).

Reducing the number of transistors will first of all reduce silicon area.

Furthermore, the dynamic power (56) will also be reduced when reducing the number of transistors. As mentioned earlier, $C_L$ is an effective capacitance that must be charged and discharged when a signal changes value. More specifically $C_L$ is a capacitive load associated with the transistor gate (Weste & Harris, 2010, p. 42). Energy is required to charge and discharge $C_L$ and thus dynamic power is consumed when each transistor in a circuit is actively switching. Fewer transistors mean lower total capacitance and thus lower switching energy.

Lastly, reduction in number of transistors will also reduce static power. An OFF transistor will leak a small amount of current $I_{static}$ which results in a static power dissipation (Weste & Harris, 2010, p. 43), given by

$$P_{static} = I_{static} V_{DD} \tag{58}$$

where $V_{DD}$ is the voltage that is supplied if the transistor is ON.

The number of transistors needed for circuit implementation of the boolean expressions directly from Table 6 is shown in Table 7, where the number of transistors in three-input AND and three-input OR ports is 8 transistors. This is because a three-input AND port must be implemented using a three-input NAND port with an inverter, and a three-input OR port must be implemented using a three-input NOR port with an inverter (Weste & Harris, 2010, p. 10).

Table 7: Overview of estimated transistor counts.

| **Signal** | **Expression** | **Transistors** |
|---|---|---|
| $\alpha$ | $A\overline{B}\,\overline{C} + AB\overline{C} + A\overline{B}C$ | $3 \times \text{AND}_3 + \text{OR}_3 = \mathbf{32}$ |
| $D_A$ | $B\overline{C}\,\overline{\beta} + A\overline{C}\beta + A\overline{B}C$ | $3 \times \text{AND}_3 + \text{OR}_3 = \mathbf{32}$ |
| $D_B$ | $\overline{A}\,\overline{B}C\overline{\beta} + AB\overline{C}\beta + A\overline{B}\,\overline{C}\,\overline{\beta} + A\overline{B}C\beta$ | $4 \times \text{AND}_4 + \text{OR}_4 = \mathbf{50}$ |
| $D_C$ | $\overline{A}\,\overline{B}\,\overline{C}\,\overline{\beta} + \overline{A}B\overline{C}\beta + AB\overline{C}\,\overline{\beta} + A\overline{B}\,\overline{C}\beta$ | $4 \times \text{AND}_4 + \text{OR}_4 = \mathbf{50}$ |
| Inverters for $A, B, C, \beta$ | $\overline{A}, \overline{B}, \overline{C}, \overline{\beta}$ | $4 \times \text{INV} = \mathbf{8}$ |
| **Total** | $\mathbf{32 + 32 + 50 + 50 + 8 = 172}$ | |

*Assumptions:* $\text{INV} = 2$, $\text{AND}_3 = 8$, $\text{AND}_4 = 10$, $\text{OR}_3 = 8$, $\text{OR}_4 = 10$ transistors. Input inversions ($\overline{A}, \overline{B}, \overline{C}, \overline{\beta}$) are generated once and shared.

Switching to NAND gates will be more power efficient because they avoid all the extra inverters that is needed to create AND gates. By applying De Morgan's law (Aayushi, 2025), each expression from Table 6 can be rewritten from AND/OR combinations to only NAND combinations. Below is an example on how it is done for $D_A$.

$$D_A = \overline{\overline{B\overline{C}\,\overline{\beta} + A\overline{C}\beta + A\overline{B}C}} = \overline{\overline{B\overline{C}\,\overline{\beta}} \cdot \overline{A\overline{C}\beta} \cdot \overline{A\overline{B}C}} \tag{59}$$

Table 8 shows results after some boolean algebra and applying De Morgan's law on each expression, with transistor count reduction.

Table 8: Overview of reduced estimated transistor counts.

| Signal | Expression | Transistors |
|---|---|---|
| $\alpha$ | A | $0 \times \text{NAND}_3 = \mathbf{0}$ |
| $D_A$ | $\overline{\overline{B\overline{C}\,\overline{\beta}} \cdot \overline{A\overline{C}\beta} \cdot \overline{A\overline{B}C}}$ | $4 \times \text{NAND}_3 = \mathbf{24}$ |
| $D_B$ | $\overline{\overline{\overline{A}\,\overline{B}C\overline{\beta}} \cdot \overline{AB\overline{C}\beta} \cdot \overline{A\overline{B}\,\overline{C}\,\overline{\beta}} \cdot \overline{A\overline{B}C\beta}}$ | $5 \times \text{NAND}_4 = \mathbf{40}$ |
| $D_C$ | $\overline{\overline{C} \cdot (\overline{\overline{A}\,\overline{B}\,\overline{\beta}} \cdot \overline{\overline{A}B\beta} \cdot \overline{AB\,\overline{\beta}} \cdot \overline{A\overline{B}\,\beta})}$ | $6 \times \text{NAND}_4 + 1 \times \text{INV} = \mathbf{50}$ |
| Inverters for $A, B, C, \beta$ | $\overline{A}, \overline{B}, \overline{C}, \overline{\beta}$ | $4 \times \text{INV} = \mathbf{8}$ |
| **Total** | $\mathbf{0 + 24 + 40 + 50 + 8 = 122}$ | |

*Assumptions:* INV = 2, $\text{NAND}_3 = 6$, $\text{NAND}_4 = 8$ transistors. Input inversions ($\overline{A}, \overline{B}, \overline{C}, \overline{\beta}$) are generated once and shared.

Using the simplified equations, the final logic diagram for the combinational part from Figure 21 was constructed. Figure 22 shows the logic diagram for the combinational part.

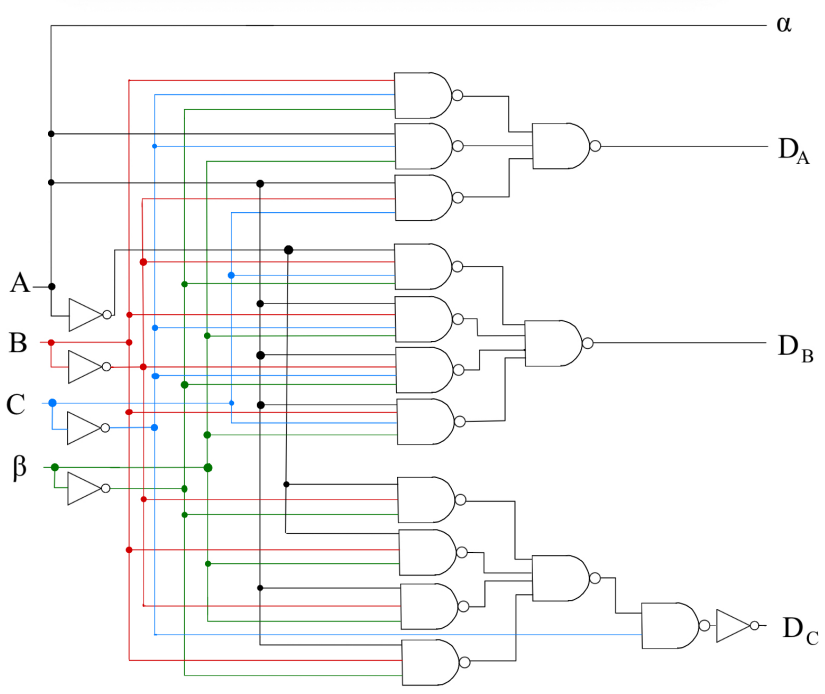


Figure 22: Logic diagram of the combinational part of the general state machine.

The combinational part, in the figure above, is then combined with a sequential part to form the final finite state machine. The sequential part is built using master-slave D flip-flops, which is an edge triggered circuit. From here and out, a master-slave D flip-flop will be referred to either DFF6NAND or simply flip-flop. As shown in Figure 23, the circuit contains only NAND gates.

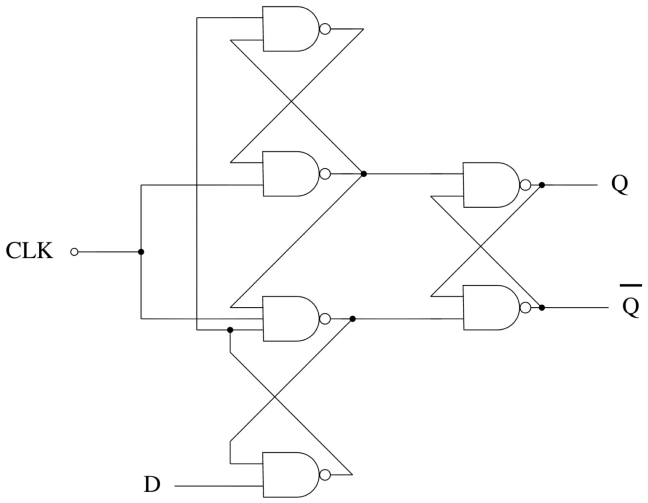


Figure 23: Circuit diagram of master-slave D flip-flop.

The circuit above corresponds to storage elements of the finite state machine. One of such flip-flop will store one bit of either present state $A$, $B$ or $C$. Three of these circuits are needed to store all three bits. Figure 24 shows the circuit symbol of the flip-flop.

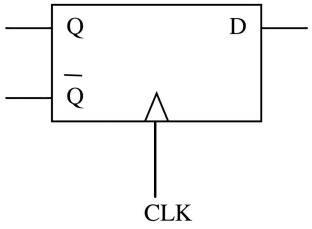


Figure 24: Circuit symbol of the flip-flop

Using the figure of the flip-flop symbol above, and the logic schematic of the combinational part the general state machine can be drawn as shown in Figure 25 below.

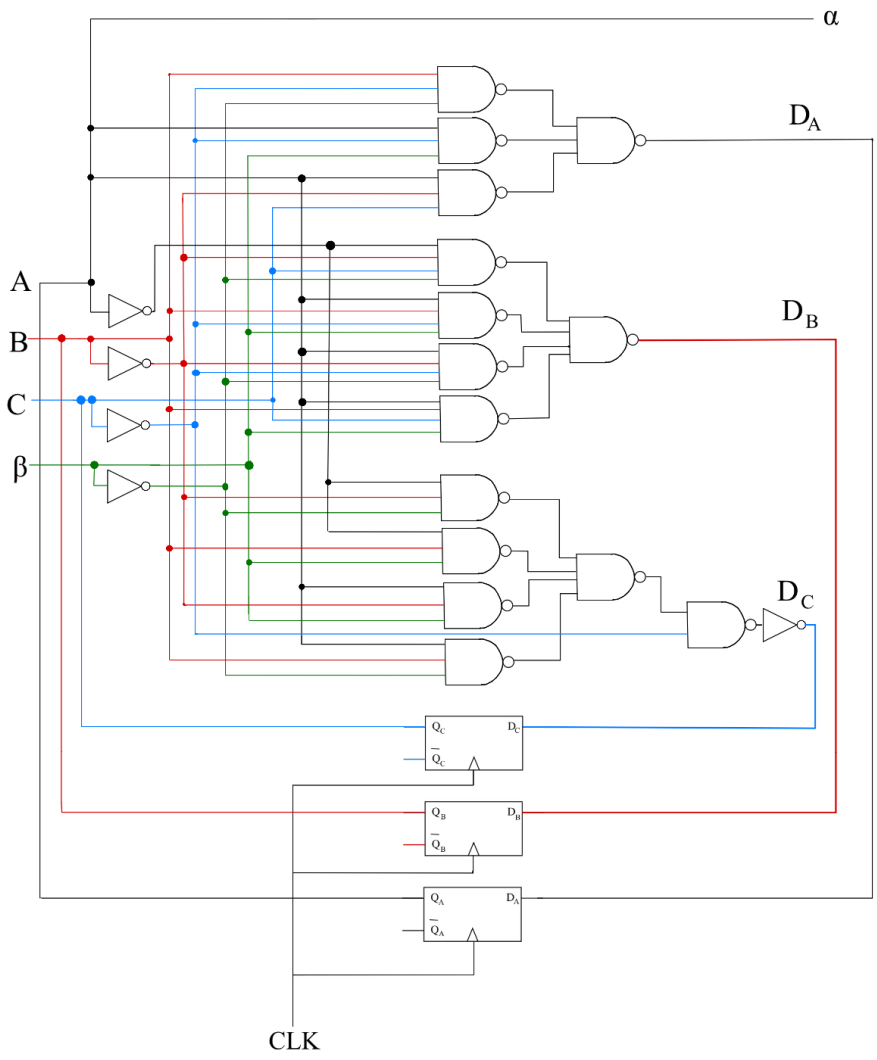


Figure 25: Logic diagram of the final finite state machine.

The diagram shows the connection between the flip-flops and the combinational logic that generates their input signals. The output logic, determined solely by the present state, is also shown. A code implementation of the finite state machine can be done in Verilog to verify that it works as expected.

## 3.2 Implementing the State Machine

A digital implementation of the finite state machine in Figure 25 is done in Verilog. Figure 21 shows that the finite state machine can be implemented as two separate blocks. One block being the combinational part and one block being the sequential part. Implementing the finite state machine, as a combination of the separate blocks, allows testing the two systems by themselves before combining them to the final finite state machine. ActiveHDL was used for Verilog programming.

### 3.2.1 Combinational part of the Finite State Machine

The code below shows how the combinational part of the finite state machine is implemented in Verilog. In short, it takes present state bits $A$, $B$, $C$ and input $\beta$. It then produces the output $\alpha$ as well as the three flip-flop input signals $D_A$, $D_B$, and $D_C$.

Listing 2: Verilog implementation of the combinational FSM logic.

```
1  module COMBIN ( b ,A ,B ,C ,a ,D_A ,D_B ,D_C );
2
3  input b;
4  wire b;
5  input A;
6  wire A;
7  input C;
8  wire C;
9  input B;
10 wire B;
11 output a;
12 wire a;
13 output D_A;
14 wire D_A;
15 output D_B;
16 wire D_B;
17 output D_C;
18 wire D_C;
19
20 //}} End of automatically maintained section
21
22 wire NA, NB, NC, Nb;
23 not (NA, A);
24 not (NB, B);
25 not (NC, C);
26 not (Nb, b);
27
28 assign a = A;
29
30 wire O1, O2, O3;
31 nand DA1( O1, A, NB, C );
32 nand DA2( O2, B, NC, Nb );
33 nand DA3( O3, A, NC, b );
34 nand DAF( D_A, O1, O2, O3 );
35
36 wire O4, O5, O6, O7;
```

```
37 nand DB1( O4, NA, NB, C,  Nb );
38 nand DB2( O5, A,  NB, NC, Nb );
39 nand DB3( O6, A,  NB, C,  b  );
40 nand DB4( O7, A,  B,  NC, b  );
41 nand DBF( D_B, O4, O5, O6, O7 );
42
43 wire O8, O9, O10, O11;
44 nand DC1( O8, NA, NB, Nb );
45 nand DC2( O9, NA, B,  b  );
46 nand DC3( O10, A,  NB, b  );
47 nand DC4( O11, A,  B,  Nb );
48
49 wire s;
50 nand DCOR( s, O8, O9, O10, O11 );
51
52 wire dc_and_s;
53 nand DCL1( dc_and_s, NC, s );
54 not (D_C, dc_and_s);
55
56 endmodule
```

The combinational part is implemented in its own module, `COMBIN`, and the logic level code directly describes the logic schematic of the combinational part of the finite state machine. First, all the inputs and outputs of the combinational part are defined, where `a` and `b` corresponds to $\alpha$ and $\beta$. Then, the inverted versions of all the inputs are created and declared as $NA = \overline{A}$, $NB = \overline{B}$, $NC = \overline{C}$, $Nb = \overline{\beta}$.

Implementation of the output $\alpha$ and the flip-flop input signals are done with the help of looking at both Figure 25 and each of their respective boolean expressions from Table 8. Output $\alpha$ is logic 1 simply when $A$ is logic 1 and is directly implemented as

$$\texttt{assign } a = A.$$

Because we are implementing the code in Verilog based off of Figure 22, which is designed with low power consumption in mind, the implementation in Verilog also considers low power consumption.

### 3.2.2 Sequential part of the Finite State Machine

The two code blocks below shows how the sequential part of the finite state machines is built. The code below shows how a single flip-flop is implemented in Verilog, based on the circuit in Figure 23.

Listing 3: Verilog implementation of a single D flip-flop.

```
1  module DFF6NAND ( D ,CLK ,Q ,QN );
2
3  input D;
4  wire D;
5  input CLK;
6  wire CLK;
7  output Q;
8  wire Q;
9  output QN;
10 wire QN;
11
12 //}} End of automatically maintained section
13
14 // Enter your statements here //
15 nand G1(O1, O4, O2);
16 nand G2(O2, O1, CLK);
17 nand G3(O3, O2, CLK, O4);
18 nand G4(O4, O3, D);
19 nand G5(Q, O2, QN);
20 nand G6(QN, Q, O3);
21
22 endmodule
```

As seen in both the code above and in Figure 23, the circuit is built by using only NAND gates. The variable names of the inputs and outputs in the code corresponds to those in Figure 24. The module is called DFF6NAND as the master-slave flip-flop is a D flip-flop that is realized with 6 NAND ports. As mentioned earlier, the code above is an implementation of a single D flip-flop, thus it only stores one bit. The code below shows how an implementation of three flip-flops forms the register that is able to store all three bits $A$, $B$ and $C$.

Listing 4: Three-bit register implemented using three DFF6NANDs.

```
1  module REG3DFF6NAND ( D1 ,D2 ,D3 ,CLK ,Q1 ,Q1N ,Q2 ,Q2N ,Q3 ,Q3N );
2
3  input D1;
4  wire D1;
5  input D2;
6  wire D2;
7  input D3;
8  wire D3;
9  input CLK;
10 wire CLK;
11 output Q1;
12 wire Q1;
13 output Q1N;
14 wire Q1N;
15 output Q2;
```

```
16 wire Q2;
17 output Q2N;
18 wire Q2N;
19 output Q3;
20 wire Q3;
21 output Q3N;
22 wire Q3N;
23
24 //}} End of automatically maintained section
25
26 // Enter your statements here //
27 DFF6NAND DFF1(D1, CLK, Q1, Q1N);
28 DFF6NAND DFF2(D2, CLK, Q2, Q2N);
29 DFF6NAND DFF3(D3, CLK, Q3, Q3N);
30
31 endmodule
```

The module above forms the whole sequential register, that is the second block in Figure 21. As seen above, the code simply contains inputs and outputs in addition to three instances of the DFF6NAND moduile. Each instance stores stores either $A$, $B$ or $C$. The clock, `CLK`, synchronizes the flip-flops and ensures state transitions on positive clock edge.

### 3.2.3 Final Finite State Machine

The Verilog implementation of the complete finite state machine is shown below.

Listing 5: Top-level Verilog implementation of the finite state machine

```
1  module TsetlinMachine (a);
2
3  reg b;
4  reg CLK;
5  output a;
6  wire a;
7
8  //}} End of automatically maintained section
9
10 // Enter your statements here //
11
12 COMBIN C1( b, A, B, C, a, D_A, D_B, D_C );
13 REG3DFF6NAND C2( D_A, D_B, D_C, CLK, A, Q1N, B, Q2N, C, Q3N );
14
15 endmodule
```

The code above shows that the finite state machine is implemented by combining the sequential part and the combinational part into a fully Moore type finite state machine. In this implementation, $\beta$ and `CLK` are declared as **reg** because we want to test the Tsetlin Machine for specific values at specific clock edges later. In this finite state machine, the combinational part will generate next-state and output signals based on current state. The register will store the present state and update on each clock edge.

### 3.3 Verification Plan for the State Machine

Table 9 shows the verification plan for the state machine to test that it works as expected. The combinational part and the sequential part are simulated separately before simulating the whole state machine as one. This allows potential design errors to be spotted and corrected before testing the whole system.

Table 9: Verification plan for the state machine.

| **Test-case** | **Circuit** | **Description** | **Stimuli applied** | **Expected output(s)** |
|---|---|---|---|---|
| T0 | Seq. | Verify correct storage and propagation of bits through flip-flops. | Apply CLK (0→1→0 with 100 ns period), toggle D-inputs (0→1→0 with 200 ns period). | $Q$ follows $D$ on rising clock edge, for all values of $D$ |
| T1 | Combin. | Verify logic corresponds to Table 5. | Test for all combinations of $A, B, C, \beta$. | Outputs $\alpha$, $D_A$, $D_B$, $D_C$ match Table 5. |
| T2 | FSM | Verify correct traversal of state diagram 17 from left to right ($\phi_0$ to $\phi_5$) | Force $(A, B, C) = 000$ for one clock cycle, then release. Apply sequence $\beta = 0, 0, 0, 1, 1, 1$ with one clock cycle per value. | We expect states to traverse from $\phi_0$ to $\phi_5$, following the values in Figure 18. |
| T3 | FSM | Verify correct traversal of state diagram 17 from right to left ($\phi_5$ to $\phi_0$). | Force $(A, B, C) = 110$ for one clock cycle, then release. Apply sequence $\beta = 0, 0, 0, 1, 1, 1$ with one clock cycle per value. | We expect states to traverse from $\phi_5$ to $\phi_0$, following the values in Figure 18. |

The table above summarizes how the finite state machine was tested to verify that it behaves as intended. The first testcase T0 tested the sequential part. Testing was done in the `DFF6NAND` module where the periodic signals of CLK and $D$ were applied in the waveform window of ActiveHDL directly. The CLK signal with period 100 ns was generated, where the values go from logic 0 to

logic 1 in 50 ns, then logic 1 to logic 0 the next 50 ns. The $D$ signal with 200 ns was generated, where the values go from logic 0 to logic 1 in 100 ns, then logic 1 to logic 0 the next 100 ns. Observations were done on the output $Q$ was done in the waveform window to verify that $Q$ updated on rising edge of the clock and followed $D$. This test was chosen to ensure that the flip-flop was implemented correctly. If the flip-flop is implemented correctly then it is safe to assume that all the flip-flops in `REG3DFF6NAND` stores and propagates bits correctly.

Next, testcase T1 tested the combinational part. Testing was done in `COMBIN` module. Similarly to above, the inputs $(A, B, C, \beta)$ were stimulated in waveform. The stimulation was done by having $\beta$ be periodic with 100 ns, as explained for CLK. Furthermore, $C$ was set to have a 200 ns period, $B$ a 400 ns period, and $A$ a 800 ns period. This doubling of the periods leads to all combinations of the inputs $(A, B, C, \beta)$ being applied systematically, following Table 5. Then observations were done on the outputs $\alpha$, $D_A$, $D_B$ and $D_C$. Each combination of input, which resulted in a combination of output, were compared to the theoretical values from the binary coded state table 5. Matching values for all input combinations will verify that the behaviour of the combinational part is as expected. This test was necessary to confirm that both the logic implementation of the combinational part and the boolean expressions were correct.

Finally, the final finite state machine was tested by combining the sequential part with the combinational part. Test T2 tested the traversal of the state diagram 17 from left to right and ensured it was correct. This was done by forcing the state bits $(A, B, C)$ to be 000 to initialize that the finite state machine to start in state $\phi_0$. The values were then released and $\beta$ was given the sequence $0, 0, 0, 1, 1, 1$, with each value holding for one clock period. Observations were done in the waveform window to check that the states were transitioning from $\phi_0$ to $\phi_5$. Same procedure was done for T3, but this time starting from $(A, B, C) = 110$. $\beta$ was again applied with the same sequence to verify correct traversal from $\phi_5$ to $\phi_0$. The testbench code from both T2 and T3 is shown below.

Listing 6: Clock generation and stimulus for the TsetlinMachine testbench

```
initial begin
    CLK = 1;
    forever #1 CLK = ~CLK;
end

initial begin
    // TEST FROM LEFT TO RIGHT
    // Force A, B and C to 000 (state  0 )
    force A = 0;
    force B = 0;
    force C = 0;
    b = 0; #2;

    // Release A, B and C after two clock cycles
    release A;
```

```
16      release B;
17      release C;
18
19      b = 0; #2;
20      b = 0; #2;
21      b = 1; #2;
22      b = 1; #2;
23      b = 1; #2;
24      b = 1; #2;
25
26      // TEST FROM RIGHT TO LEFT
27      // Force A, B and C to 110 (state  5 )
28      force A = 1;
29      force B = 1;
30      force C = 0;
31      b = 0; #2;
32
33      // Release A, B and C after two clock cycles
34      release A;
35      release B;
36      release C;
37
38      b = 0; #2;
39      b = 0; #2;
40      b = 1; #2;
41      b = 1; #2;
42      b = 1; #2;
43      b = 1; #2;
44
45      $finish;
46  end
```

Testing of the Tsetlin Machine, as shown above, does minimal testing of what happens if the finite state machine ends up in a 'don't care' state. Only minimal testing is done as there is no implementation of a solution of what is going to happen if the state machine ends up in a 'don't care' state. 'Don't care' states are the states that are marked with x in Table 5. The minimal testing is done by checking, when transitioning from left to right and right to left, that we never end up in a 'don't care' state. Some final tests of the 'don't care' states are done in each ends. When at the end, $\phi_0$ and $\phi_5$, the testcase gives one last reward, $\beta = 1$ and checks if the state machine remains in the same state.

If an implementation of a solution to what happens when the state machine ends up in 'don't care' was implemented, a potential test could be to stimuli the finite state machine to start in the 'don't care' states and check if the solution works. A potential solution could be that if the state machine ends up in 'don't care', for example $\phi_x = 011$, then the state machine either subtracts or add one bit such that the machine transitions to either $\phi_2$ or $\phi_3$. Similar solution can be implemented if state machine ends up in $\phi_x = 111$

As mentioned earlier, T0 and T1 were chosen to test the sequential part and combinational part separately to spot potential errors, and then allow to correct the errors before combining the systems. T2 and T3 will validate that the combined finite state machine works as expected by comparing theoretically expected behaviours to the observed behaviours of the machine. Together, these tests verifies that the finite state machine operates correctly and as wished.

## 3.4 Results from the State Machine Tests

The verification plan from section 3.3 was followed. Table 10 shows a summary of expected results compared to observed results and if the testcase passed or failed.

Table 10: Results from state machine testing.

| **Testcase** | **Expected output(s)** | **Observed output(s)** | **Pass/Fail** |
|---|---|---|---|
| T0 | $Q$ follows $D$ on rising clock edge. | $Q$ follows $D$ on rising clock edge. | Pass |
| T1 | Outputs $\alpha$, $D_A$, $D_B$, $D_C$ match Table 5. | Outputs $\alpha$, $D_A$, $D_B$, $D_C$ match Table 5. | Pass |
| T2 | State bits $(A, B, C)$ follow expected sequence and there is correct traversal from $\phi_0$ to $\phi_5$. | State bits $(A, B, C)$ follow expected sequence and there is correct traversal from $\phi_0$ to $\phi_5$. | Pass |
| T3 | State bits $(A, B, C)$ follow expected sequence and there is correct traversal from $\phi_5$ to $\phi_0$. | State bits $(A, B, C)$ follow expected sequence and there is correct traversal from $\phi_5$ to $\phi_0$. | Pass |

The table above shows that all the testcases passed and the observed output was just as expected for every testcase. Below are some of the test results, starting with Figure 26 showing the results from testcase T0.

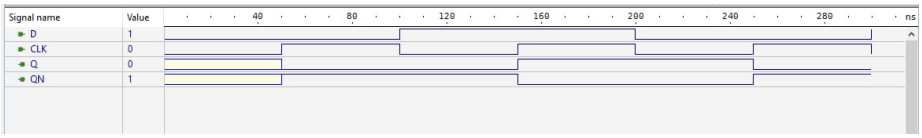


Figure 26: T0 (sequential part) result.

By following the clock signal CLK, we are able to observe that $Q$ reflects the current value of $D$ for every positive clock edge. Moving on to the results of the combinational part, Figure 27 shows the results from testcase T1.

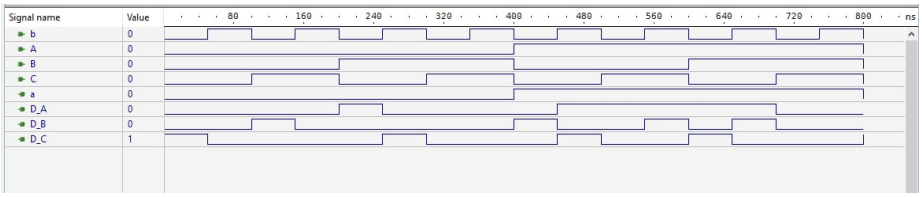


Figure 27: T1 (combinational part) result.

By looking at each combination of input $A$, $B$, $C$ and $\beta$ and their corresponding combination of output $D_A$, $D_B$, $D_C$ and $\alpha$, then comparing to Table 5, we are able to confirm that the combinational part matches Table 5. Moving on to the results of the combination of the sequential part and the combinational part, Figure 28 and Figure 29 shows the results from testcase T2 and T3.

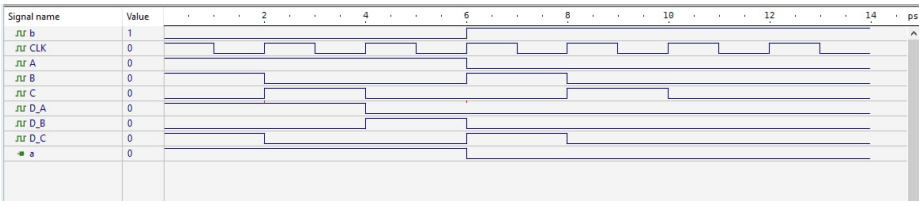


Figure 28: T2 (tsetlin machine, left to right) result.

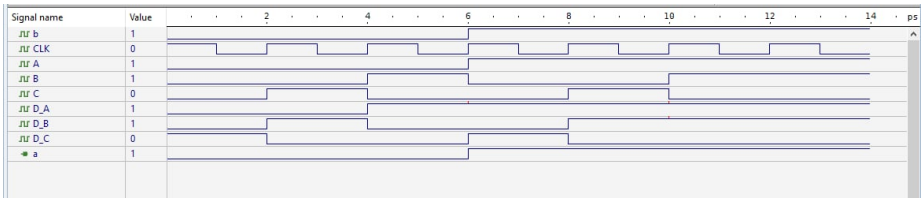


Figure 29: T3 (tsetlin machine, right to left) result.

The current state given by $A$, $B$ and $C$, and the input given by $\beta$, yields the correct outputs $D_A$, $D_B$, and $D_C$, as shown in Table 5. Further, we see that each positive clock edge leads to the state $A$, $B$ and $C$ becoming the computed next state of $D_A$, $D_B$ and $D_C$, following Figure 18. As explained, T2 is made to follow Figure 18 from left to right, which Figure 28 confirms it does. T3 is made to follow Figure 18 from right to left, which Figure 29 confirms it does.

## 3.5 Discussion

One of the specifications for the finite state machine was that it should be designed with low power consumption in mind. For our implementation of the finite state machine, many design choices were made to specifically reduce both switching activity and unnecessary logic. Thus, reducing overall power consumption.

Firstly, minimizing bit toggling activity, as explained in section 3.1 step 3, contributes to reducing power consumption. The activity factor $\alpha$ from the formula for dynamic power (56) was reduced by observing that average bit flips could be reduced if $\phi_3 = 011$ is $\phi_3 = 100$ instead. Number of bit flips from $\phi_3 = 011$ to $\phi_4 = 100$ is three, compared to two bit flips after reduction. Number of bit flips when transitioning from $\phi_3 = 100$ and outwards are the same both before and after minimizing bit toggling. Calculating how much power is reduced can be done by calculating the activity factor (Weste & Harris, 2010, p. 187).

Secondly, logic simplification by looking at transistor level of the logic gates in step 7 also contributed to reducing power consumption. By simplifying the boolean expressions as much as possible by using Karnaugh maps to find expressions reduced the amount of logic gates needed for implementation. With further boolean algebra and De Morgan's law it was possible to reduce the amount of transistors even more by using only NAND gates for implementation of the combinational part of the finite state machine. For example, output $\alpha$ was substantially reduced such that no gates were needed to implement the output. With logic simplification, both dynamic power and static power was reduced. As explained in section 3.1, logic simplification reduced dynamic power because of the reduction in number transistors which leads to reduction in load capacitance which is proportional to dynamic power. While static power is reduced because current leakage is reduced, which is proportional to static power.

Lastly, as seen from the logic diagram of the combinational part, shown in Figure 22, the input signals have been inverted only once, instead of being inverted every time an inverted signal is needed for the different NAND gates.

Even though several choices were made to design the finite state machine with low power consumption in mind, there are many other improvements that can be done that could help lower the power consumption. Improvements can be done to reduce both the static power and dynamic power.

**Reducing Static Power**

As explained above, logic simplification will reduce power consumption. Further logic simplification for the circuit in Figure 22 is possible. One example is to reuse gates. For example, $D_A$ contains the term $\overline{A\overline{B}C}$, which is also seen in $D_B$. By reusing this term from $D_A$ for $D_B$ we would save one four-input NAND port in favor of one two-input NAND port. Which is a reduction of 4 transistors. By finding more terms that can be reused will lead to further reduction in logic gate implementation, which as discussed, reduces static power consumption.

Reduction in power consumption can also be achieved by using compound gates. Compound gates are also static CMOS logic, which is how we have implemented our NAND gates. The advantage of compound gates is that complex boolean expressions are collapsed into one single gate, and thus reduces transistor count (Weste & Harris, 2010, p. 11).

Another approach to lower power consumption is to use multiple threshold voltages. One contributor to leakage current is subthreshold leakage that flows even when a transistor is OFF. Subthreshold leakage $I_{sub}$ is dependent on threshold voltage $V_t$, where high threshold devices limits power leakage (Weste & Harris, 2010, p. 119). Equation for $I_{sub}$, which is shown below (Carusone, et al., 2011, p. 43), shows how subthreshold current is dependent on $V_t$.

$$I_{\text{sub}} = I_{D0} \left( \frac{W}{L} \right) e^{\frac{-qV_t}{nkT}} \tag{60}$$

For an explanation of all variables in the equation, see page 43 of (Carusone et al., 2011, p. 43). Increasing the channel length of the transistors is an approach to increasing $V_t$. Longer channel will reduce short channel effects which will lead to an increase in $V_t$ (Weste & Harris, 2010, p. 199). In addition longer channel will lower the leakage. As $L$ is the channel length of a transistor, the subthreshold leakage is shown to be inverse proportional to it in (60). A tradeoff to consider, when increasing $V_t$, is how it will affect how fast a transistor switches ON. A larger $V_t$ requires higher gate voltage before a transistor turns ON, as we need $V_{GS} > V_t$ for a transistor to turn ON. Thus, larger $V_t$ makes a transistor switch more slowly.

Another approach to reducing subthreshold leakage is stacking transistors, in series of two or more. This approach exploits the stack effect and will also contribute to reducing the static power consumption (Weste & Harris, 2010, p. 195).

### Reducing Dynamic Power

An approach to reduce dynamic power is voltage scaling. As seen in equation for dynamic power (56), the supply voltage is squared and thus has a quadratic effect. Dynamic voltage scaling lowers the power supply during periods of reduced performance requirements (Weste & Harris, 2010, p. 191). This approach is not relevant for our finite state machine. Clustered voltage scaling on the other hand, divides a circuit into different blocks that runs on different levels of supply voltage (Weste & Harris, 2010, p. 191). This allows certain blocks to operate with the power that they need, while other blocks operate with reduced supply voltage. The blocks that operate with reduced supply voltage are not the critical path. Overall $V_{DD}$ is reduced which furthermore reduces dynamic power $P_{dyn}$.

To summarize, many design choices were made to reduce overall power consumption. For example, minimizing bit toggling activity and logic simplification. Static power could have been reduced even more by considering other approaches such as using compound gates, using multiple threshold voltages, increasing channel lengths of transistors or stacking transistors. Dynamic power on the other hand could have been reduced with clustered voltage scaling.

# References


[1] Carusone, T.C., Johns, D.A., Martin, K.W. (2011). *Analog Integrated Circuit Design* (2nd ed.). John Wiley & Sons.

[2] Weste, N.H.E., Harris, D.M. (2010). *CMOS VLSI Design* (4th ed.). Pearson Education.

[3] Mano, M.M. (1991). *Digital Design* (2nd ed.). Prentice Hall.

[4] Kartik (2025, September 08). *Introduction of K-Map (Karnaugh Map)*. GeeksforGeeks. https://www.geeksforgeeks.org/digital-logic/introduction-of-k-map-karnaugh-map/

[5] Aayushi (2025, July 23). *De Morgan's Law - Theorem, Proofs, Formula & Examples*. GeeksforGeeks. https://www.geeksforgeeks.org/maths/de-morgans-law/

[6] AIM-Spice (2020, June 24). *AIM-Spice*. http://www.aimspice.com/

[7] Learning Microelectronics (n.d.). *Analog IC Design Spice Models & Examples*. https://analogicdesign.com/students/netlists-models/